ProbTest: Unit Testing for Probabilistic Programs

Katrine Christensen, Mahsa Varshosaz, and Raúl Pardo(✉)

IT University of Copenhagen, Copenhagen, Denmark
raup@itu.dk

Abstract. Testing probabilistic programs is non-trivial due to their stochastic nature. Given an input, the program may produce different outcomes depending on the underlying stochastic choices in the program. This means testing the expected outcomes of probabilistic programs requires repeated test executions unlike deterministic programs where a single execution may suffice for each test input. This raises the following question: how many times should we run a probabilistic program to effectively test it? This work proposes a novel black-box unit testing method, *ProbTest*, for testing the outcomes of probabilistic programs. Our method is founded on the theory surrounding a well-known combinatorial problem, the *coupon collector's problem*. Using this method, developers can write unit tests as usual without extra effort while the number of required test executions is determined automatically with statistical guarantees for the results. We implement *ProbTest* as a plug-in for PyTest, a well-known unit testing tool for python programs. Using this plug-in, developers can write unit tests similar to any other Python program and the necessary test executions are handled automatically. We evaluate the method on case studies from the Gymnasium reinforcement learning library and a randomized data structure.

1 Introduction

Probabilistic programs are programs with stochastic behavior. They have been long used in several domains such as the implementation of randomized algorithms [4,26,27,31], and are at the core of widespread machine learning systems [7,24,28,36]. Probabilistic programs may return different outcomes for the same input due to their stochastic behavior. Hence, the output of a probabilistic program induces a probability distribution over possible program outcomes. This stochastic behavior makes testing probabilistic programs notoriously difficult.

As an example, consider the program in Listing 1.1. It implements a one dimensional discrete random walk. Although this program looks simple, random walks are the core of multiple probabilistic inference methods such as Markov Chain Monte Carlo (MCMC) [8]. Consider a developer whose task is to test whether it is possible for this program to assign a value larger than a boundary b to the (returned) variable `pos`. This property may be useful, for instance, to detect integer overflow bugs. Evidently, to check this property the developer

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
D. Bianculli and E. Gómez-Martínez (Eds.): SEFM 2025, LNCS 16192, pp. 91–109, 2026.
https://doi.org/10.1007/978-3-032-10444-1_6

```
def n_random_walk(n: int):
    pos = 0 # init_pos
    X = bernoulli(1/2) # from scipy.stats
    for _ in range(n):
        x_sample = X.rvs()
        if x_sample == 1
            pos = pos + 1
        if x_sample == 0
            pos = pos - 1
    return pos
```

Listing 1.1. Python implementation of a discrete one dimensional random walk of `n` steps.

must run the program repeatedly. However, this leads to the following question: *"How many times is it necessary to run the program to detect a violation of this property?"* Answering this question is not a trivial task. The challenge illustrated in this simple example transcends to many domains and large software systems with probabilistic behavior. For instance, how many times should we execute a test to assert a property in a randomized data structure, in a probabilistic protocol, or in a policy generated by a Reinforcement Learning (RL) algorithm?

Existing work in this field tackles this problem by using a model of the program [21–23,30,33], such as a Markov Decision Process [3]. However, this requires white-box access to the program and its libraries, which is not always available. It also requires to generate a model from the program, which creates a gap between the actual program and the model. Other works [12,17] propose to find bugs by performing statistical tests between the output distribution of the program and a distribution of the target property. Unfortunately, applying and interpreting the results of these methods often requires expertise in statistics. For example, testers must be able to provide a distribution for the target property; or at least its mean and variance. In practice, testers resort to running probabilistic programs an arbitrary number of times; hoping to trigger existing bugs, but without any guarantees.

In this work, we present a novel black-box unit testing method with statistical guarantees, named *ProbTest*, to effectively test probabilistic programs. As it is black-box, the tester does not need to know the internal structure of the program. Testers specify an assertion (as in regular unit testing), and the probability of violating the assertion, i.e., the expected failure probability. An example for the property and program above is *"The probability that the program in listing 1.1 returns a value greater than a boundary b is at most 10%"*. *ProbTest* is proven to ensure that, if a bug exists in the program, an assertion failure is triggered with probability $1 - \epsilon$; where $\epsilon > 0$ is an error probability selected by the tester. To this end, *ProbTest* automatically (and efficiently) computes the

number of unit test executions (k) needed to trigger an assertion violation with probability $1-\epsilon$. Thus, our method guarantees that if all k runs of the test pass, the property holds with probability $1-\epsilon$. If a test run fails, the property does not hold for the program, and we have discovered a bug. As is a unit testing method, the complexity of properties that *ProbTest* handles is naturally lower than traditional verification techniques such as statistical model-checking. However, the number of executions k required by *ProbTest* can be orders of magnitude lower than for statistical model-checking or testing methods based on statistical tests (cf. Sect. 5). The underlying theory for *ProbTest* is based on the well-known *coupon collector's problem* [29]. In summary, our contributions are:

1. *ProbTest*, a method for unit testing of probabilistic programs with statistical guarantees (Sect. 3).
2. We establish the correctness of the method for testing probabilistic programs (Sect. 3), and study its scalability and sensitivity to errors.
3. An implementation of the method as a plug-in for the PyTest framework that can be used for unit testing of probabilistic programs in Python.
4. An empirical evaluation of the effectiveness of *ProbTest* on reinforcement learning applications and a randomized data structure (Sect. 4). The evaluation demonstrates the effectiveness of *ProbTest* to discover bugs in probabilistic programs used in large systems and practical applications.

The source code of *ProbTest*, the PyTest plugin and experiments are available in the accompanying artifact [11].

2 Background

2.1 Probabilistic Programs

Probabilistic programs are programs with stochastic behaviour in their execution. That is, programs can make a random choice at any point in their execution; which may result in producing random outcomes for the same input. In this paper, we use the following formal definition for probabilistic programs:

Definition 1. *Given an input $i \in I$, a probabilistic program f is described by the probability space $(\Omega_i, \mathcal{F}_i, P_i)$, where Ω_i is the set of all outcomes of the program for input i, $\mathcal{F}_i$ is a σ-algebra such that $\mathcal{F}_i \subseteq 2^{\Omega_i}$ and $P_i \colon \mathcal{F}_i \to [0,1]$ is a probability measure. We use $\mathcal{O}_i$ to denote the set of outcomes/outputs of the program (the terms outcome and output are used interchangeably throughout the paper). Finally, we use $O_i : \Omega_i \to \mathcal{O}_i$ to denote a random variable for the observable outcomes of the program. A probabilistic program thus induces a probability distribution over the outcomes of the program for a given input i. When no confusion arises, we will denote $(\Omega_i, \mathcal{F}_i, P_i)$ as $(\Omega, \mathcal{F}, P)$ and O_i as O.*

The definition above views the execution of the program as a random experiment; i.e., sampling from O_i. Our definition is a simplification of the standard pushforward measure over program statements [4] where we only consider the

distribution of the program output for a given input. Focusing on the program output distribution is sufficient, given our back-box unit testing setup. We only consider positively almost-surely terminating programs, i.e., programs that terminate with probability 1 and finite expected runtime [4]. Requiring termination in black-box unit testing is a common restriction, as assertions are checked after the execution of the program. Note that the probability space of the program only depends on $i \in I$. Thus, in a sequence of samples drawn from O_i, each sample is independent and identically distributed. We will use $f(i)$ to denote the probabilistic program f for an input $i \in I$. Note that $f(i)$ is not a function. Furthermore, the set of inputs I is an abstraction. It can represent any combination of input parameters to a program.

Example 1. Consider the program in Listing 1.1, which implements a one dimensional discrete random walk. The program simulates taking n random steps, each step moving either left or right based on a coin toss. For simplicity, let us consider a random walk consisting of only two steps, i.e., `n = 2`. The set of possible outcomes is $\mathcal{O} = \{-2, \ldots, 2\}$ and the σ-algebra $\mathcal{F}$ is the powerset of the set of outcomes $\mathcal{F} = 2^{\mathcal{O}}$. The output random variable O is a distribution defined as $P(O = -2) = P(O = 2) = 1/4$ (as ± 2 are only reachable after obtaining two consecutive 0 s or 1 s from `X.rvs()`), $P(O = 0) = 1/2$ (as 0 can be reached by obtaining first 0 and then 1 or first 1 and then 0 from `X.rvs()`), and $P(O = 1) = P(O = -1) = 0$ (as ± 1 are not possible outcomes after two steps).

2.2 Coupon Collector's Problem

The coupon collector's problem is a well-known combinatorial problem [29]. It concerns the problem of drawing balls with replacement from an urn with a finite collection of N different balls until each ball is drawn at least once. The balls are drawn with probability $p_i \in (0, 1]$ with $\sum_{i=1}^{N} p_i \leq 1$. When $\sum_{i=1}^{N} p_i \neq 1$, there are one or multiple balls in the urn that are not of interest to the collection.

Consider the random variable T for the number of draws with replacement to obtain a full collection of the N different balls. As T depends on the probability distribution of the balls, we denote it as $T(\mathbf{p})$ where $\mathbf{p} := (p_1, \ldots, p_N)$ with p_i representing probability of drawing ball i. The problem is determining the number of balls $k \in \mathbb{N}$ that need to be drawn to obtain a full collection with high probability. Formally, the problem is formulated as $P(T(\mathbf{p}) > k)$; i.e., the tail distribution of finding all balls after k trials. The following theorem establishes an exact solution for the tail distribution (e.g., [35]).

Theorem 1. *Let $k > 0$, then $P(T(\boldsymbol{p}) > k)$ is given by*

$$P(T(\boldsymbol{p}) > k) = \sum_{i=1}^{N} (-1)^{i+1} \sum_{J:|J|=i} (1 - P_J)^k \tag{1}$$

where $P_J := \sum_J p_i$ and J is a subset of $\{1, ..., N\}$. That is, the inner sum is a sum over all elements of size i of the power set of $\{1, ..., N\}$.

We map the problem of testing probabilistic programs to the coupon collector's problem and use its theoretical results to develop our unit testing method.

3 Unit Testing of Probabilistic Programs

Here we introduce our unit testing method for probabilistic programs. We consider unit tests that assert a Boolean property after the execution of the program under test for a given input. These are standard in unit testing. An assertion is a map from program outcomes to Boolean values $Q\colon \mathcal{O} \to \{0,1\}$ where 0 and 1 denote false and true, respectively. Composing the assertion with the random variable for the outcome of program under test $Q \circ O$ results in a new random variable over Boolean values; as the assertion is a measurable map [18].

The stochastic nature of probabilistic programs requires a probabilistic notion for passing or failing a unit test. To this end, we introduce the notion of *assertion coverage* for probabilistic programs. Intuitively, we require that the unit test is executed enough times so that program outcomes satisfying/violating the assertion are induced during executions with high probability. This ensures that, with high probability, one execution of the unit test will produce an outcome violating the assertion, if there exists such outcome (e.g., in case there is a bug in the program). The following definition formalizes this notion.

Definition 2 (Assertion Coverage for Probabilistic Programs). *Let $f(i)$ denote a probabilistic program under test f for input $i \in I$ described by the probability space $(\Omega, \mathcal{F}, P)$. Let $O\colon \Omega \to \mathcal{O}$ be the random variable for $f(i)$. Let $Q\colon \mathcal{O} \to \{0,1\}$ be a measurable map corresponding to an assertion (i.e., a Boolean property on program outcomes) for the program under test. Let T be a random variable for the number of times to run the program to see all assertion outcomes, i.e., $\{0,1\}$. Given $\epsilon > 0$ and $k > 0$, we say that a unit test achieves assertion coverage after executing the program k times with probability $1-\epsilon$ iff*

$$P(T > k) < \epsilon. \tag{2}$$

We refer to $P(T > k)$ as the tail distribution of T.

In what follows, we present, *ProbTest*, our black-box unit testing method for probabilistic programs, and we prove that it satisfies assertion coverage.

Algorithm 1 shows the pseudo-code of our method to assert Boolean properties over outcomes of probabilistic programs. The algorithm takes four inputs. The first two inputs are: the probabilistic program under test $f(i)$ (for program input $i \in I$) and the assertion to test $Q\colon \mathcal{O} \to \{0,1\}$. These inputs are standard in unit testing. Additionally, the method takes as input an *assertion specification* vector $\mathbf{p} = (p, 1-p)$, where $p \in (0,1)$ is the *assertion violation probability.* This parameter specifies the maximum allowed probability for the assertion to be violated. For instance, a requirement for the program can be that *"property Q must hold with probability 0.99"*. In this case, the assertion violation probability must be set to $p = 0.01$, and, consequently, $\mathbf{p} = (0.01, 0.99)$. The last input is the error probability $\epsilon > 0$, which indicates the accuracy of the test finding a violation if one exists. Lower values of error probability result in higher accuracy.

The last two input parameters p and ϵ, result from the stochasticity of the probabilistic program. Although these parameters are not standard in regular

Algorithm 1. Black-box unit testing method pseudo-code.

Input: Input value $i \in I$; Program under test $f(i)$ described by $(\Omega_i, \mathcal{F}_i, P_i)$ Assertion $Q\colon \mathcal{O} \to \{0,1\}$; Assertion spec. $\mathbf{p} = (p, 1-p)$ with $p \in [0,1]$; and Error probability $\epsilon > 0$

Output: Unit test result $r \in \{0,1\}$; pass (1) or fail (0)

1: **procedure** PROBTEST$(i, f(i), Q, \mathbf{p}, \epsilon)$
2: $\quad k \leftarrow \arg\min_{k \in \mathbb{N}} P(T(\boldsymbol{p}) > k) < \epsilon$ ▷ *To be determined using Theorem 1*
3: $\quad$ **for** 1 to k **do**
4: $\quad\quad o \xleftarrow{\$} f(i)$ ▷ *The symbol* $\xleftarrow{\$}$ *denotes sampling an output from* $f(i)$
5: $\quad\quad$ **if** $\neg Q(o)$ **then**
6: $\quad\quad\quad$ **return** 0
7: $\quad\quad$ **end if**
8: $\quad$ **end for**
9: $\quad$ **return** 1
10: **end procedure**

unit testing, they commonly appear in the analysis of probabilistic systems (cf. Sect. 5). We remark that, even though p and ϵ cannot be set to 0, they can be as small as needed. This is a decision that the tester takes based on the requirements for the program under test. In Sect. 3.1, we study the scalability and accuracy trade-offs of selecting different values for these parameters.

Algorithm 1 uses Theorem 1 to automatically determine the number of times (k) that the unit test must be executed to achieve assertion coverage with probability less than $1-\epsilon$ (line 2). In our implementation, we incrementally explore values of k starting from $k = 2$ until $P(T(\boldsymbol{p}) > k) < \epsilon$ holds. This naive search exhibits good practical performance. For instance, iterating up to values of k in the order of 10^7 takes less than 2 s. However, more efficient methods to compute arg min can be used for this step of the algorithm. In lines 3–8 we run the probabilistic program up to k times. Line 4 denotes executing the probabilistic program. If the output does not satisfy the assertion, we stop the execution of the unit test and return false (line 6); as we have found an error in the program. If after k executions we find no program output violating the assertion, we return true (line 9). This indicates that the assertion is satisfied with probability $1-\epsilon$.

Example 2. Consider again the `n_random_walk` program in Listing 1.1, and the property stating that the value of `pos` does not exceed a boundary b with certain probability. Here we discuss how to apply *ProbTest* to test this property.

In this example, $f(i)$ is the probabilistic program `n_random_walk` and the program input, `n=20`, determines the number of steps of the random walk. The assertion Q is a Boolean predicate checking whether the position after executing the 20 steps is below some boundary b. We arbitrarily use $b = 10$ for this example. The following listing shows the code for the complete unit test:

```
def test(n: int = 20, b: int = 10):
    o = n_random_walk(n) # f(i)
    assert o < b # Q
```

To determine how many times this test must be executed, we define the assertion specification $\mathbf{p}$ and probability of error ϵ. We illustrate two possibilities for a tester to decide on the values for these parameters: based on system requirements or based on expected program behavior.

For the system requirements case, assume high level system requirements that specify the required robustness of the system. For example, *"with probability 0.99 the random walk must not finish in a position more than 10 steps away from the origin."* Probabilistic requirements on the length of a random walk are commonplace in MCMC algorithms for Bayesian inference [8]. To check this requirement, the tester must set $\mathbf{p} = (0.01, 0.99)$. We use a value of $\epsilon = 0.02$, which defines the accuracy of the method. This value of ϵ means that if the probability for the random walk to finish 10 steps away from the origin is more than 0.01, then a violation of the property will be triggered with probability 0.98 (i.e., $1-\epsilon$) if one exists. A tester may choose arbitrarily lower values of ϵ (as long as they are larger than 0) to increase accuracy; as expected, lower ϵ values result in larger number of executions of the test (cf. Sect. 3.2). For $\mathbf{p} = (0.01, 0.99)$ and $\epsilon = 0.02$, our algorithm requires to execute the test $k = 390$ times.

The tester can also select $\mathbf{p}$ and ϵ based on their knowledge about the program. For example, the documentation may state that the random walk moves to adjacent states with probability 1/2. Thus, it is a possible to finish in a position greater than 10 with probability $1/2^{10}$. To determine whether this is a possible behavior in the program, the tester sets $\mathbf{p} = (1/2^{10}, 1 - 1/2^{10})$ and the same accuracy as before $\epsilon = 0.02$ In this scenario, the test must be executed $k = 4004$. Note that reducing the probability of violating the assertion (from 0.01 to $1/2^{10}$) requires increasing the number of executions (cf. Sect. 3.2). As before, this number of executions guarantees that if the assertion can be violated with probability $1/2^{10}$, then with probability 0.98 we will observe a program outcome violating the property in one of the $k = 4004$ executions. □

The correctness of *ProbTest* follows from the coupon collector problem.

Theorem 2. *Let $f(i)$ be a probabilistic program with input $i \in I$, $\boldsymbol{p}$ be an assertion specification, and $\epsilon > 0$ the probability of error. Let $k \in \mathbb{N}$ be the number of times Algorithm 1 executes the test (defined in line 2), and $T(\boldsymbol{p})$ be the random variable for the required number of program (test) executions to observe all outcomes of $f(i)$. Then, $P(T(\boldsymbol{p}) > k) < \epsilon$, i.e., Algorithm 1 achieves test assertion coverage for $f(i)$ with probability $1 - \epsilon$.*

Proof. The theorem follows by mapping the elements of the unit testing method in Algorithm 1 to those of the coupon collector's problem. The set of balls maps to the set of property outcomes ($\{0, 1\}$), the distribution for drawing each ball maps to the distribution of property outcomes ($\mathbf{p}$), and each draw (with replacement) maps to an execution of the unit test (k). Since Algorithm 1 uses Theorem 1 to determine a value of k such that $P(T(\mathbf{p}) > k) < \epsilon$ and it runs the test k times, then it directly follows that test assertion coverage (Definition 2) holds with probability $1 - \epsilon$, as required. □

3.1 Scalability of Test Executions

Computational resources may impact the assertion violation probability p or error probability ϵ values that can be used in practice. Although the value of k can be computed efficiently using Theorem 1 (since N is the main factor determining the complexity of computing k and we are in the case for $N = 2$), the value of k increases rapidly as the values of p and ϵ decrease; especially for low p, ϵ values. In other words, as the tester sets higher requirements for the correctness of the assertion, the method returns a increasingly larger k. Figure 1 shows the rapid growth of k for $p, \epsilon \in (0.001, 0.1)$. We observe, for instance, that if a test can be executed at most 60 times then the accuracy $(1 - \epsilon)$ can be at most 97.6% and the assertion violation probability 0.056. However, executing unit tests thousands of times is commonplace, and the plot shows that with less than 7000 test executions we achieve 99.9% accuracy for an assertion violation probability of 0.001. In Sect. 4 we demonstrate that *ProbTest* can be effectively used in realistic applications.

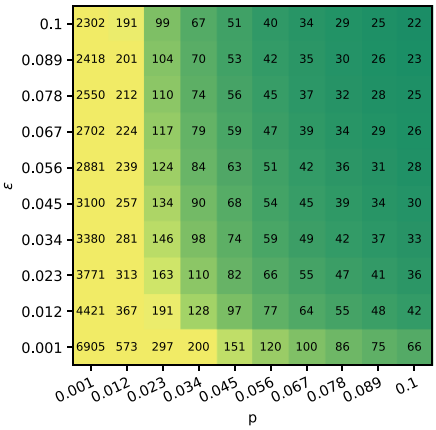

Fig. 1. Values of k for $p, \epsilon \in (0.001, 0.1)$

3.2 Sensitivity to Error in Assertion Violation Probabilities

Let $\hat{p}$ denote an assertion violation probability, for an assertion Q, that differs from the real assertion violation probability p given by the program implementation. In this situation, a natural question is: *What is the probability that our method triggers an outcome violating the assertion if $\hat{p}$ is used instead of p in Algorithm 1?* The answer depends on whether $\hat{p}$ under- or overestimates p.

If a tester underestimates the value of p, i.e., $\hat{p} < p$, then, for any ϵ, the value of k may be larger than necessary. This is due to the monotonic increase of k for decreasing values of $\epsilon > 0$ and $p \in (0, 0.5]$. This trend is symmetric for $p > 0.5$ as $1 - p < 0.5$ (cf. Theorem 1 for $N = 2$). Thus, our method will require a value $\hat{k}$ such that $\hat{k} \geq k$ (where $\hat{k}$ and k correspond to $\hat{p}$ and p, respectively). But the impact of using $\hat{k}$ is minimal. Algorithm 1 finishes after an assertion violation is found, and Theorems 1 and 2 show that this must happen in at most k executions. Therefore, our algorithm will repeat the test only k times. The only overhead of using $\hat{p}$ is that it takes longer to find $\hat{k}$, as it is a larger value than k. Since determining $\hat{k}$ can be done efficiently, this overhead is minimal.

If the tester overestimates the assertion violation probability, i.e., $\hat{p} > p$, then the value of k will be smaller than necessary; again due to the monotonic increase of k for decreasing p, ϵ. This implies that the probability of executing a violating outcome will be smaller than if p had been used instead of $\hat{p}$. This is problematic, as it detriments the chances of the tester finding errors in the program under test.

In Fig. 2, we explore the effect of overestimating p on the probability of executing a violating outcome. We consider the effect of overestimating p by a factor ϕ (i.e., $\hat{p} = p + p \cdot \phi$), for different values of p and for different error probabilities ϵ. For each case (cells in the figure) we show the difference in number of executions $k - \hat{k}$ and the difference between the tail probability and the specified probability of error $P(T(\mathbf{p}) > \hat{k}) - \epsilon$. The latter quantifies the decrease in the probability of executing an outcome violating the assertion if $\hat{p}$ is used instead of p. We consider small values p and ϵ, as this is where more variation occurs (cf. Sect. 3.1).

For a very small overestimation $\phi = 0.01$ (first column in Fig. 2), we observe very small difference in number of executions $(k - \hat{k})$ for all $\epsilon \in \{0.001, 0.01\}$, with only notable changes in the case $p = 0.001$. Despite this difference, note that the difference between tail probability and probability of error is 0 for all cases. This means that overestimating p by a factor of $\phi = 0.01$ has no impact on the probability of executing a violating outcome.

We observe in Fig. 2 that the distance between number of executions $k - \hat{k}$ heavily depends on the value of p for all ϵ values. Lower values of p result in higher distance between number of executions as ϕ increases. This is expected due to the growth in the value of k for small values of p and ϵ that we observed in Fig. 1. However, the large difference between number of executions does not necessarily imply a large decrease in executing a violating outcome.

Interestingly, Fig. 2 shows that the distance between the tail probability and ϵ is very similar (and often the same) for all columns. This indicates that changes on the probability of executing a violating outcome are mainly affected by the error factor ϕ, and not by the value of p. Nevertheless, the value of ϵ affects the magnitude of the distance between the tail probability and ϵ. For instance, in order for the distance to be ≈ 0.10 (i.e., it is around 10% less likely to execute the outcome violating the assertion), the error factor can be up to $\phi \leq 1.67$ for $\epsilon = 0.001$ and up to $\phi \leq 1.12$ for $\epsilon = 0.01$. This effect generalizes to any distance, i.e., using smaller ϵ values always produces smaller decrease on the probability of executing a violating outcome for the same error factor ϕ. This insight reveals that, if testers are unsure about the value of p, it is recommended to select an ϵ value that is as small as possible; as this ensures a higher probability of finding errors in case $\hat{p}$ is overestimating p.

Implementation. We have implemented *ProbTest* as a plugin for the Python testing framework *Pytest* [11,13]. To apply the plugin, the tester must provide: i) the system under test as a Python program; ii) a test suite for the system under test for an input written in *Pytest* (the test suite may consist of a finite number of tests, each testing a property of the system); iii) an assertion specification $\mathbf{p} = (p, 1 - p)$ where $p \in (0, 1)$ is the assertion violation probability; and iv) the error probability $\epsilon > 0$. The plugin automatically determines the number of times to execute each test in the test suite and runs the tests sequentially. The execution finishes as soon as one test fails; as described in Algorithm 1.

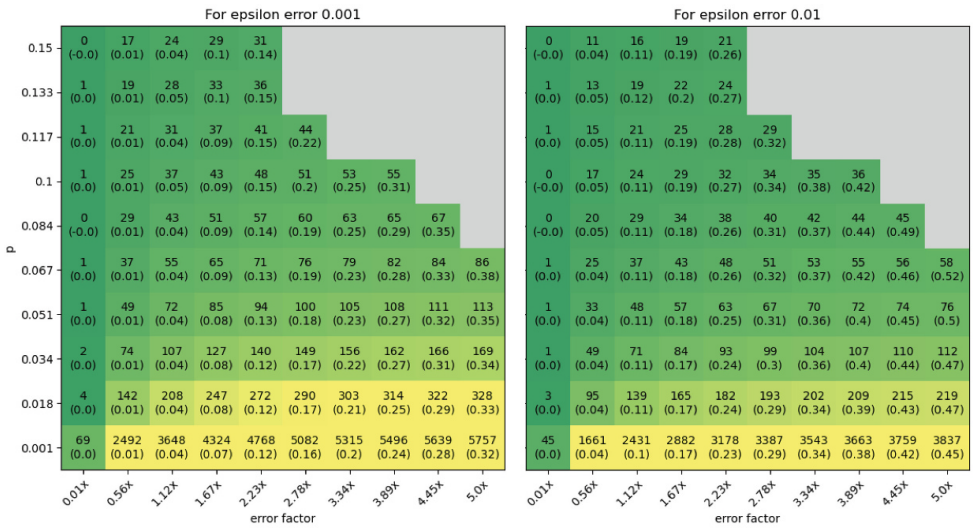

Fig. 2. Effect of overestimating p on the value of k and the probability of finding a violating outcome. The y-axis is the real assertion violation probability p. The x-axis displays the factor of error, ϕ, for the overestimated assertion violation probability, $\hat{p} = p + p \cdot \phi$. Each cell has two rows: the upper row shows $k - \hat{k}$ (corresponding to p and $\hat{p}$, respectively), and the bottom row shows the distance between the tail probability for $\hat{k}$ and the error probability, $P(T(\mathbf{p}) > \hat{k}) - \epsilon$. We explore 2 error probabilities $\epsilon \in \{0.001, 0.01\}$. Gray cells corresponds to cases with $\hat{p} > 0.5$. These are symmetric to $1 - \hat{p}$ and are already covered in the plot, as $1 - \hat{p} < 0.5$.

4 Case Studies

In this section, we demonstrate using *ProbTest* in two realistic case studies: a randomized data structure (Sect. 4.1) and a reinforcement learning system (Sect. 4.2). The source code and results for all experiments are available in [11].

4.1 Skip List

A *skip list* is a randomized data structure consisting of a finite collection of ordered linked lists of nodes $L_1, \ldots, L_M$ [27]. We write $n \in L_i$ to denote list L_i contains node n. Each L_i denotes a *level*. Each level contains a subset of the nodes from the level below, i.e., $L_M \subseteq L_{M-1} \subseteq \ldots \subseteq L_1$. Thus, the first level L_1 contains all nodes in the skip list, and the size of the skip list is defined as $R = |L_1|$. A node in L_i has a pointer (`next`) to the next node in the link list, and, if $i > 1$, a pointer (`lower`) to a node with the same key in the level below L_{i-1}. The level of a node n is defined as $\max(\{i \mid n \in L_i\})$. Figure 3 depicts an example skip list with 2 nodes and levels L_1, L_2, L_3. Node 1 has level 1 and node 2 level 3. The symbol $\bullet$ denotes the head of each L_i and `None` is the `next` value for nodes at the tail. One of the benefits of skip lists is the expected time $O(\log(R))$ for insert, delete and

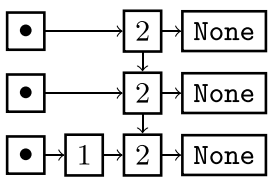

Fig. 3. Skip list with 2 nodes and 3 levels.

search [27]. Various industrial applications use skip lists (e.g., [15,34]). Due to the randomized nature of skip lists, standard unit testing may be ineffective. We use *ProbTest* to test a skip list implementation (with statistical guarantees), and demonstrate its applicability for testing software used in industry.

The randomized behavior of skip lists originates in insertions. When inserting a node, the node is first added to L_1 in its correct place according to the order among nodes. Then, a coin with bias p_r is tossed. If the outcome is heads, the node is added to the layer above (also preserving order). Otherwise, the insertion finishes. The coin toss is repeated until the outcome is tails. Consequently, executing an insertion produces a set possible skip lists; depending on its randomized behavior.

To test our skip list implementation, we have designed a battery of 26 unit tests. The test are split into 4 categories: validity, post-condition, metamorphic and equivalence. Validity tests check that after executing an insertion, deletion or search the skip list remains in a valid state. According to [27], a skip list is valid if the following properties hold: i) the maximum number of layers is never exceeded; ii) each layer is a subset of the layer below it; iii) all layers are ordered; iv) all inserted elements should be in the bottom layer; v) No node in L_1 have pointers to nodes below them; and vi) Nodes in L_i with $i > 1$ must have a pointer the layer below L_{i-1} if it contains a node with the same key. Post-condition tests check that operations behave as expected. For instance, after deleting a node, a search for said node returns `None`. Metamorphic properties encode properties between two lists that must hold before and after executing a set of operations. For example, consider two lists l_1, l_2 that contain the same nodes. One of our metamorphic tests checks that after adding a new node to l_1 and l_2 the lists still contain the same elements—although they may be structurally different due to randomness. Finally, equivalence properties are similar to metamorphic properties for list equivalence but also include properties checking cases where the resulting lists must not be equivalent. For instance, consider again l_1, l_2. If we add an element only to l_1, then l_1 and l_2 must not contain the same nodes. The accompanying artifact contains the details of each of the 26 properties [11].

The next step is to define the assertion violation probability p. The goal is to select a value of p ensuring that all possible skip lists resulting from a series of insertions are tested at least once. To this end, we determine the probability of generating the most unlikely skip list after a series of insertions. This ensures that we will check the properties above for this unlikely skip list (with high probability). Also, since any other resulting skip list have higher probability, they will be covered by at least one execution (cf. Sect. 3.2).

The probability of generating the most unlikely skip list depends on the number of possible skip lists and the probability of raising a node. Since each node can be raised up to M levels, then there are M^R possible skip lists after R insertions. Let p_r denote the probability of raising a node. When $p_r \leq 1/2$, the least likely state is when all R nodes are raised to the maximum layer. When $p_r > 1/2$, however, the least likely state is when all nodes are raised to the layer just below the maximum layer, L_{M-1}. This corresponds to raising the node

repeatedly $M-2$ times with probability $p^{(M-2)}$, followed by a final coin toss with the result of not raising the node with probability $(1-p)$. In summary, we define the assertion violation probability as

$$p = \begin{cases} p_r^{R(M-1)} & \text{if } p_r \leq 1/2 \\ p_r^{R(M-2)}(1-p_r)^R & \text{otherwise.} \end{cases} \tag{3}$$

To evaluate the effectiveness of *ProbTest*, we injected 8 bugs in our skip list implementation. The bugs are designed to appear for an increasing number of insertions $R = 1..4$. There are two bugs for each R. The goal is to evaluate the ability of *ProbTest* to find bugs that appear with decreasing probability. Each bug is unique, and they may be triggered by one or many of the skip lists that result from running a test. For instance, one of the injected bugs fails to delete nodes at the top layer. This bug occurs only in one of the possible skip lists that result after 1 insertion. Another injected bug incorrectly raises the level of a set of nodes to L_{i+1} if there are 3 or more nodes at level L_i. In this case, the bug may be triggered by more than one of the resulting skip lists resulting from 3 insertions. We refer interested readers to [11] for details of each bug.

Experiments and Results. We consider a skip list with node raising probability $p_r = 1/2$ and at most $M = 3$ levels, the battery of 26 unit tests and the 8 injected bugs. Given this setup and Eq. (3), the assertion violation probability (p) equals $0.25, 0.0625, 0.0156, 0.0039$ for R equals 1 to 4, respectively. We consider a probability of error $\epsilon = 0.05$. We run each experiment 1000 times and estimate the probability that *ProbTest* finds the injected bugs. As a sanity check, we first run the experiments with no injected bugs for 1–4 insertions. As expected, *ProbTest* finds no bugs. Figure 4 shows the results for injected bugs. The first two columns show bug ID and the number of insertions required to trigger the bug. The third column shows the number of times that *ProbTest* executes the test. The last column shows the estimated probability of *ProbTest* finding the bug. We can observe that, for most bugs, *ProbTest* finds the bugs with probability ≥ 0.99. Only for bugs 3 and 7 the probability decreases to 0.955 and 0.96, respectively. These bugs corresponds to those that are only triggered with very low probability. Nevertheless, all bugs are detected with probability at least 0.95, i.e., $1-\epsilon$. These results are a consequence of Theorem 2 and selecting a value of p that is lower or equal to the probability of generating the skip list with lowest probability after R insertions.

bug	R	k	Prob. find bug
1	1	11	1.000
2	1	11	0.997
3	2	47	0.955
4	2	47	1.000
5	3	191	0.997
6	3	191	1.000
7	4	767	0.960
8	4	767	1.000

Fig. 4. Estimated probability of finding injected bugs with *ProbTest* (1000 executions).

All in all, we demonstrated the use of *ProbTest* to effectively test a realistic randomized data structure that is used in industrial applications, namely, the skip list. Furthermore, our results show that *ProbTest* finds bugs with the desired statistical guarantees.

4.2 Reinforcement Learning Applications

We consider two RL case studies from the Gymnasium Python library [37]; one of the most widely used libraries to develop RL agents in Python. Gymnasium environments serve as testbeds for RL algorithms.

Table 1. Left to right: assertion violation probability in specification (p), number of training episodes, estimated probability that the property fails over 100k runs, probability that *ProbTest* finds failure over 100 runs.

	p	Property	Num. episodes	Prob. property does not hold	Prob. Probtest finds bug
frozen lake 4×4	0.01	Q1	100	0.000	0.000
frozen lake 4×4	0.001	Q2	100	0.002	0.990
frozen lake 7×7	0.01	Q1	25000	0.000	0.000
frozen lake 7×7	0.001	Q2	25000	0.060	1.000
cliff walking	0.001	Q	25000	0.007	1.000

Frozen Lake. The environment involves an $n \times m$ board that represents a frozen lake with holes distributed on the lake. An agent can move around the board until it falls into a hole or reaches a goal state. The holes and goal states are terminal states. We consider two board sizes 4×4 and 7×7 provided in the Gymnasium library documentation. The board has stochastic behavior representing slippery ice, which means the agent moves in the intended direction with a probability of 1/3, and (slipping) in one of the two perpendicular directions with a combined probability of 2/3.

Cliff Walking. This is another classic RL example in which an agent must navigate a map while avoiding falling off a cliff that spans the edge of the map. If the agent steps into the cliff area, it receives a negative reward and is sent back to the initial state. The environment is episodic and includes stochastic dynamics.

Reinforcement learning agents learn a policy to achieve a goal by interacting with an environment using RL algorithms. This process involves training the agent through repeated episodes, where it explores actions and updates its behavior based on feedback from the environment. We use Q-learning [39], a traditional RL algorithm, which updates a Q-table to approximate the optimal state-action-values (find the best action) based on observed transitions and rewards during training. The algorithm implementation uses a standard exploration strategy in which actions are selected at random with a small configurable probability and otherwise the best action is selected according to the policy.

Experiments and Results. We are interested in studying whether we can discover errors in the behavior of an agent trained in these stochastic environments. Due to the randomness in the environment, traditional unit testing methods are not applicable for this type of systems. Here we demonstrate the use of *ProbTest* to test reinforcement learning applications. The experiments are designed to assess whether *ProbTest* can be used to test reinforcement learning policies. For the case studies, a *policy* determines the actions (movements) of agent, and the *environment* determines the state (position) of the agent given a selected action. Note that source of randomness is in the environment, but it could also be in the policy due to the small probability of selecting a random action as explained above. Since our method is black-box, the exact source of randomness is irrelevant. We do not directly inject bugs in the program, as the lack of enough training is considered to be the source of buggy behavior in the experiments (and in practical applications). For the frozen lake we test the following properties: *Q1: The agent never falls into holes in s steps following policy* π, and *Q2: The agent never takes more than s steps before reaching a terminal state following policy* π. For s we use values 100 and 200 for the 4×4 and 7×7 boards (these values are selected based on the episode lengths recommended by the Gymnasium library documentation). To test these properties using *ProbTest*, we must provide an assertion specification for each program property. Unlike for the skip list, here we consider that the assertion violation probability is defined by high-level requirements. This type of probabilistic high-level requirements are common in practical applications of RL. Let $P(\neg Q1)$ and $P(\neg Q2)$ denote assertion violation probabilities for $Q1$ and $Q2$, respectively. We consider $P(\neg Q1) = 0.01$ and $P(\neg Q2) = 0.001$. That is, *"the agent must not fall into holes with probability 0.99"* and *"the agent must reach a terminal state before 100 steps with probability 0.999"*, respectively. These values have been chosen arbitrarily small on the basis that we expect the agent to be faster at learning to avoid holes than taking too many steps. Similarly for the cliff walking, we test the property *Q: The agent survives without falling off the cliff for at least s steps*, with $s = 30$ and $P(\neg Q) = 0.001$. The value of the parameters are selected proportionally to the map size of cliff walking in the Gymnasium library.

A summary of the results of experiments is presented in Table 1. First, we train each RL agent for a number of episodes (third column in Table 1) selected considering the Gymnasium documentation. Then, we estimate the probability of property violations by evaluating the policy for 100k runs (column 4 in Table 1). This value is used to approximate the distance between the real probability of a violation to the value of p that we selected for $Q1$ and $Q2$. Finally, we run *ProbTest* using the assertion violation probabilities above and $\epsilon = 0.05$ (as before). *ProbTest* executes the test 299 times for property $Q1$ and 2995 times for the other properties. The last column in Table 1 shows the estimated probability of *ProbTest* finding a violation (computed over 100 runs). We observe that *ProbTest* finds violations with very high probability (≥ 0.99) when the estimated probability of failure is above the assertion violation probability. This is a notably high accuracy given our $\epsilon = 0.05$, which only requires an accuracy

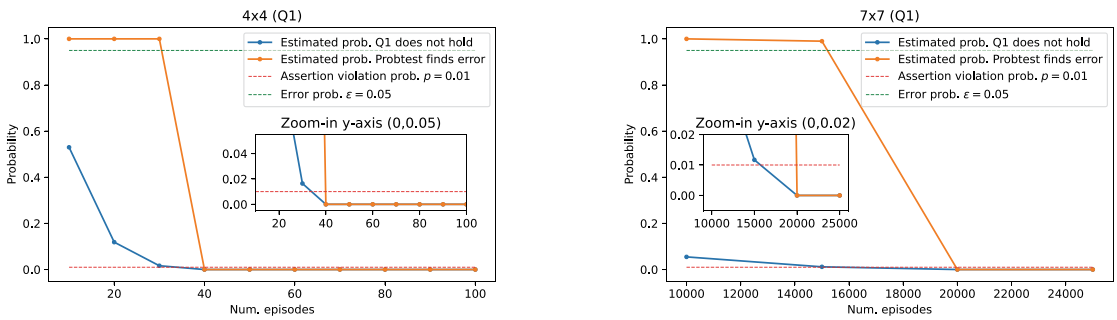

Fig. 5. Estimated policy violation probability for Q1 (blue) and probability of finding errors (orange) for the frozen lake problem using policies trained on increasing number of episodes. The red dashed line marks the assertion violation probability p and the green line $1 - \epsilon$. (Color figure online)

≥ 0.95. Additional experiments show that results also apply for different numbers of training episodes. Figure 5 plots the results for increasing number of training episodes for the frozen lake problem (both maps) and property Q1. As expected, the estimated probability that Q1 does not hold for the learned policy decreases as the number of training episodes increases. Similar to Table 1, the figure shows that *ProbTest* detects with very high probability when Q1 can be violated, given the assertion violation probability. Even in cases where the estimated probability of failure is close to the assertion violation probability (these cases are zoomed-in in the plots), *ProbTest* finds failures with probability significantly larger than $1 - \epsilon = 0.95$. The complete list of RL results is available in [11]. All in all, these experiments demonstrate the effectiveness of *ProbTest* to find violations in such RL applications.

5 Related Work and Concluding Remarks

Related Work. We discuss the most relevant works on testing probabilistic programs, but also formal verification techniques focused on probabilistic programs.

We start discussing model-based methods, i.e., methods that require a probabilistic model of the program such as a Markov Decision Process [5]. Probabilistic model checking is a formal verification technique that has been studied for many years [21,22]. Probabilistic model checking algorithms exploit algebraic methods to verify probabilistic properties [3]. These methods support expressive property languages such as PCTL [19] where assertion coverage and much more complex properties can be expressed and verified. Prasetya and Klomp use probabilistic model checking to propose a model-based method for test coverage with statistical guarantees [33]. The method can assess stricter coverage notions than assertion coverage such as path coverage. Another line of work is statistical model-checking (SMC) [23]. Given a probabilistic model, this method uses Monte Carlo simulation and statistical tests to verify whether a property holds with statistical guarantees [9,25,30]. Similar to the above, SMC methods support PCTL and other expressive property languages.

The main difference between the above works and *ProbTest* is the need for a model. Obtaining models from source code is challenging and often requires introducing abstractions. As a black-box method, *ProbTest* only requires access to execute the system. As a testing method, *ProbTest* can impose notably lower requirements in the number of executions compared to SMC. For example, SMC can be used to estimate an assertion violation probability up to some error δ and with probability ϵ, formally $P(p-\delta \leq p \leq p+\delta) \geq 1-\epsilon$. A common statistical bound to tackle this problem (without a high requirement in the number of executions) is the Hoeffding bound, which states that $\delta = \sqrt{\ln(2/\epsilon)/2k}$ where k is the number of executions [25]. Let $\delta = 0.01$ and $\epsilon = 0.05$, then this bound requires > 70000 executions. This would be the required number of executions for the case studies in Sect. 4, while *ProbTest* required less than 3000 executions, i.e., an order of magnitude less. This is expected, as SMC estimates the assertion violation probability whereas *ProbTest* checks whether a given one holds. Nevertheless, it is noteworthy that, in this setup, *ProbTest* can ensure statistical guarantees with notably less executions.

Hypothesis testing has been used to statistically test properties of randomized algorithms [1]. A set of guidelines to test whether two different randomized algorithms exhibit the same statistical behavior (e.g., expectation or variance) was proposed in [1]. A black-box testing method for probabilistic programs based on hypothesis testing method has been proposed in [17]. The method allows testers to check whether the distribution of a program's output has a given expectation and variance. This allows to test more general properties than in *ProbTest*, as there are no constraints on the program output distribution. However, the method assumes that testers can specify a program output distribution, which may require more advanced proficiency in statistics compared to *ProbTest*. The method can be used to check whether an assertion holds with certain probability by using a Bernoulli distribution with mean $1-p$ where p is the assertion violation probability. But here again the required number of executions may be larger than for *ProbTest*. This method requires executing the program $k = (2\lceil -24\ln(\epsilon)\rceil + 1)n$ (with $n > 31$) to test whether that an assertion violation probability p holds with probability $1-\epsilon$ [17]. Consider $\epsilon = 0.05$ as in the case studies in Sect. 4, then the program must be executed (at least) 4433 times. This is ≈ 1000 times more than the largest k for *ProbTest* in our case studies. It is also noteworthy that for many assertion violation probabilities *ProbTest* required less than 1000 executions, which is 4 times less than this hypothesis testing method. Using the same statistical test as in [17], a method for metamorphic testing has been proposed [16]. This method does not require specifying the exact program output distribution, but requires specifying statistical metamorphic relations between inputs and outputs.

Dutta et al. propose a method to test probabilistic programming frameworks [12], in which Bayesian probabilistic models are defined (e.g., [6,10,32]). The method checks equivalence between inferred posterior distributions and analytical solutions (computed by tools like PSI [14]) or other approximations (computed using probabilistic inference methods such as MCMC). Although,

this method could be applied in our setting, it is not designed for this purpose. Finally, this is a white-box method, as opposed to our method that only requires black-box access to the program. In [38], a test harness is proposed for reinforcement learning, including also statistical tests tuned by trial and error, underscoring the need for systematic testing for such applications.

The coupon collector's problem has been applied to analyse the effectiveness and predictability of random testing [2] and scenario-based testing of autonomous driving systems [20]. The source of randomness in this line of work is on generating program inputs, but not the probabilistic behavior of the program under test. *ProbTest* could be combined with random testing to support property-based testing of probabilistic programs.

Conclusion. We present, *ProbTest*, a novel unit testing method for probabilistic programs in a black-box setting. Given an assertion specification (i.e., the probability for the probabilistic program to violate/satisfy the assertion), we show *ProbTest* achieves assertion coverage with high probability. We have evaluated *ProbTest* using two case studies and our results demonstrate that it is an effective method to test probabilistic programs, and also it is applicable to realistic software systems.

References

1. Arcuri, A., Briand, L.C.: A practical guide for using statistical tests to assess randomized algorithms in software engineering. In: ICSE 2011, pp. 1–10. ACM (2011)
2. Arcuri, A., Iqbal, M.Z., Briand, L.C.: Formal analysis of the effectiveness and predictability of random testing. In: ISSTA 2010, pp. 219–230. ACM (2010)
3. Baier, C., Katoen, J.-P.: Principles of Model Checking. MIT Press (2008)
4. Barthe, G., Katoen, J.-P., Silva, A.: Foundations of Probabilistic Programming. Cambridge University Press, Cambridge (2020)
5. Bellman, R.: A Markovian decision process. J. Math. Mech. **6**(5), 679–684 (1957)
6. Bingham, E., et al.: Pyro: deep universal probabilistic programming. J. Mach. Learn. Res. **20**, 28:1-28:6 (2019)
7. Bishop, C.M.: Pattern Recognition and Machine Learning (Information Science and Statistics). Springer, Verlag (2006)
8. Brooks, S., Gelman, A., Jones, G., Meng, X.L.: Handbook of Markov Chain Monte Carlo. CRC Press, New York (2011)
9. Budde, C.E., Hartmanns, A., Meggendorfer, T., Weininger, M., Wienhöft, P.: Sound statistical model checking for probabilities and expected rewards. In: TACAS 2025. LNCS, vol. 15696, pp. 167–190. Springer (2025)
10. Carpenter, B., et al.: Stan: a probabilistic programming language. J. Stat. Softw. **76**(1) (2017)
11. Christensen, K., Varshosaz, M., Pardo, R.: ProbTest accompanying artifact (2025). https://github.com/itu-square/probtest-pytest
12. Dutta, S., Legunsen, O., Huang, Z., Misailovic, S.: Testing probabilistic programming systems. FSE **2018**, 574–586 (2018)
13. Krekel, H., et al.: PyTest. https://docs.pytest.org/

14. Gehr, T., Misailovic, S., Vechev, M.T.: PSI: exact symbolic inference for probabilistic programs. In: CAV'16. LNCS, vol. 9779, pp. 62–83 (2016)
15. De Gregorio, D., Di Stefano, L.: Skimap: an efficient mapping framework for robot navigation. In: ICRA 2017, pp. 2569–2576. IEEE (2017)
16. Guderlei, R., Mayer, J.: Statistical metamorphic testing testing programs with random output by means of statistical hypothesis tests and metamorphic testing. In: QSIC 2007, pp. 404–409. IEEE (2007)
17. Guderlei, R., Mayer, J., Schneckenburger, C., Fleischer, F.: Testing randomized software by means of statistical hypothesis tests. In: SOQUA@FSE 2007, pp. 46–54. ACM (2007)
18. Hansen, E.: Measure Theory, 4th edn. University of Copenhagen, Department of Mathematical Sciences (2009)
19. Hansson, H., Jonsson, B.: A logic for reasoning about time and reliability. Formal Aspects Comput. **6**(5), 512–535 (1994)
20. Hauer, F., Schmidt, T., Holzmüller, B., Pretschner, A.: Did we test all scenarios for automated and autonomous driving systems? In: ITSC 2019, pp. 2950–2955. IEEE (2019)
21. Hensel, C., Junges, S., Katoen, J.-P., Quatmann, T., Volk, M.: The probabilistic model checker storm. Int. J. Softw. Tools Technol. Transf. **24**(4), 589–610 (2022)
22. Kwiatkowska, M.Z., Norman, G., Parker, D.: PRISM 4.0: verification of probabilistic real-time systems. In: CAV 2011. LNCS, vol. 6806, pp. 585–591. Springer (2011)
23. Legay, A., Delahaye, B., Bensalem, S.: Statistical model checking: an overview. In: International Conference on Runtime Verification, pp. 122–135. Springer, Berlin, Heidelberg (2010)
24. McElreath, R.: Statistical Rethinking: A Bayesian Course with Examples in R and Stan, 2nd edn. CRC Press, UK (2020)
25. Meggendorfer, T., Weininger, M., Wienhöft, P.: What are the odds? Improving the foundations of statistical model checking (2025)
26. Morgan, C., McIver, A.: pGCL: formal reasoning for random algorithms. South African Comput. J., 14–27 (1999)
27. Motwani, R., Raghaven, P.: Randomized Algorithms. Cambridge University Press, Cambridge (1995)
28. Murphy, K.P.: Probabilistic Machine Learning: Advanced Topics. MIT Press, Cambridge (2023)
29. Neal, P.: The generalised coupon collector problem. J. Appl. Probab. **45**(3), 621–629 (2008)
30. Peck, R., Olsen, C., Devore, J.: Introduction to Statistics and Data Analysis, 3rd edn. Thomson Brooks/Cole, UK (2008)
31. Di Pierro, A., Wiklicky, H.: On probabilistic CCP. In: APPIA-GULP-PRODE, pp. 225–234 (1997)
32. Abril Pla, O., et al.: PyMc: a modern, and comprehensive probabilistic programming framework in Python. PeerJ Comput. Sci. **9**, e1516 (2023)
33. Prasetya, I.S.W.B., Klomp, R.: Test model coverage analysis under uncertainty: extended version. Softw. Syst. Model. **20**(2), 383–403 (2021)
34. Prout, A.: The story behind Singlestore's skiplist indexes. Singlestore Blog (2014). https://www.singlestore.com/blog/what-is-skiplist-why-skiplist-index-for-memsql/
35. Shioda, S.: Some upper and lower bounds on the coupon collector problem. J. Comput. Appl. Math. **200**(1), 154–167 (2007)

36. Sutton, R.S., Barto, A.G., et al.: Reinforcement Learning: An Introduction. MIT Press, Cambridge (1998)
37. Towers, M., et al.:Gymnasium: a standard interface for reinforcement learning environments (2024)
38. Varshosaz, M., Ghaffari, M., Johnsen, E.B., Wąsowski, A.: Formal specification and testing for reinforcement learning. Proc. ACM Program. Lang. **7**(ICFP), 125–158 (2023)
39. Watkins, C.J.C.H., Dayan, P.: Q-learning. Mach. Learn. **8**, 279–292 (1992)